

Assignment 8.1	
Using Sorting Algorithms 2	
Course Code: CPE010	Program: Computer Engineering
Course Title: Data Structures and Algorithms	Date Performed: Sep 23, 2025
Section: CPE21S4	Date Submitted: Sep 23, 2025
Name(s): Punay, Heidee S.	Instructor: Engr. Jimlord Quejado
Answer the following questions:	
1. Explain the quick sort, shell sort, and merge sort types of sorting algorithms. Quick Sort is where we can choose an element from the array, which will be the pivot. Then, it rearranges the elements in the array so that all elements in the array that are less than the pivot are moved to the left and the greater elements are moved to the right. Shell Sort divides the array into smaller groups using the gap. It also uses an insertion sort on elements that are a certain gap apart, and reduces the gap and repeats the process until the gap becomes 1. Merge Sort's function is to divide the array into two halves until it can no longer be divided. Merges the sorted halves into one sorted array. 4. Give simple sample programs in C++ that uses the above sorting algorithms. Use a user input of 10 integer values in your example elements in an array to be sorted. Explain how the programs work. Quick Sort:	



```
1  #include <iostream>
2  #include <vector>
3  #include <algorithm> // For std::swap
4
5  // Function to partition the array around a pivot
6  int partition(std::vector<int>& arr, int low, int high) {
7      // Choosing the last element as the pivot
8      int pivot = arr[high];
9
10     // Index of smaller element
11     int i = (low - 1);
12
13     for (int j = low; j <= high - 1; j++) {
14         // If current element is smaller than or equal to pivot
15         if (arr[j] <= pivot) {
16             i++; // Increment index of smaller element
17             std::swap(arr[i], arr[j]);
18         }
19     }
20     std::swap(arr[i + 1], arr[high]); // Place pivot at its correct position
21     return (i + 1); // Return the partitioning index
22 }
23
24 // Main QuickSort function
25 void quickSort(std::vector<int>& arr, int low, int high) {
26     if (low < high) {
27         // pi is partitioning index, arr[pi] is now at right place
28         int pi = partition(arr, low, high);
29
30         // Separately sort elements before and after partition
31         quickSort(arr, low, pi - 1);
32         quickSort(arr, pi + 1, high);
33     }
34 }
35
36 int main() {
37     std::vector<int> arr = {10, 7, 8, 9, 1, 5};
38     int n = arr.size();
39
40     std::cout << "Original array: ";
41     for (int x : arr) {
42         std::cout << x << " ";
43     }
44     std::cout << std::endl;
45
46     quickSort(arr, 0, n - 1);
47
48     std::cout << "Sorted array: ";
49     for (int x : arr) {
50         std::cout << x << " ";
51     }
52     std::cout << std::endl;
53
54     return 0;
55 }
```

First was to choose the higher element in the array as the pivot and rearrange the array where all the elements in the array that are less than equal to pivot go to left and those elements that are greater than pivot go to right. It then places the pivot in its correct sorted position. i starts before the subarray which is the low - 1 then loops scans the elements in the array from low to high - 1. If $arr[j]$ is less than equal to pivot it increments i and swap $arr[i]$ with $arr[j]$. After the loop, it then proceeds to swap $arr[i + 1]$ with the pivot to put its correct position. Quick sort function, it calls the partition to find the pivot index to sort the left subarray which is the low to $pi - 1$ and also the right subarray which is the $pi + 1$ to high. Lastly the main function, initializes the vector and prints the original array. To sort the array, it calls the quicksort function and prints the sorted array.

Output:

```
Original array: 10 7 8 9 1 5
Sorted array: 1 5 7 8 9 10
```

Shell sort:

The function shellsort sorts the array in ascending order using shell sort. To get the number of the elements in the array we initialize the int n . Then there's the for loop, it starts with a gap of half the array size and it keeps dividing it by 2 until it becomes 0. This gap determines how far apart the elements are being compared. The inside for loop which is the second for loop is for each element from the gap and stores it in temp. The third loop is to shift elements that are gap positions apart if it is greater than the temp. The $arr[j]$ that equals to temp is to place the temp in its correct position. Another function, the printArray loops in the array and prints each element separated by the space. Lastly to the main function initializes the vector of integers and prints the original position or the order of the elements in the array. It then calls the shellSort function to sort the array and prints the sorted array.

Output:

```
Original array:
23 12 1 8 34 54 2 3

Sorted array using Shell Sort:
1 2 3 8 12 23 34 54
```

Merge Sort:

```
#include <iostream>
using namespace std;

// Merge two subarrays into one sorted array
void merge(int arr[], int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;

    // Create temporary arrays
    int L[n1], R[n2];

    // Copy data to temp arrays
    for (int i = 0; i < n1; i++)
```

```

        L[i] = arr[left + i];
    for (int j = 0; j < n2; j++)
        R[j] = arr[mid + 1 + j];

    // Merge the temp arrays back into arr[left...right]
    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j])
            arr[k++] = L[i++];
        else
            arr[k++] = R[j++];
    }

    // Copy remaining elements
    while (i < n1)
        arr[k++] = L[i++];
    while (j < n2)
        arr[k++] = R[j++];
}

// Recursive merge sort function
void mergeSort(int arr[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;

        // Sort first and second halves
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);

        // Merge the sorted halves
        merge(arr, left, mid, right);
    }
}

int main() {
    const int SIZE = 10;
    int arr[SIZE];

    cout << "Enter 10 integers:\n";
    for (int i = 0; i < SIZE; i++) {
        cin >> arr[i];
    }

    mergeSort(arr, 0, SIZE - 1);
}

```

```

    cout << "\nSorted array:\n";
    for (int i = 0; i < SIZE; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;

    return 0;
}

```

The function merge is to merge two sorted subarrays. left which is the arr left to arr mid and right arr mid + 1 to arr right. It then creates a temporary array the L and the R and copies the elements from the original array into the temporary arrays. Merges the two halves back into the original array in sorted order and compares the elements from L and R though smaller elements are placed into the arr k and copies the remaining elements from L or R if either of the side finishes. The function mergeSort, it splits the array into halves and sorts each half. Calls the merge () to combine the two halves. And an if else stops recursion when the subarray has one or zero elements meaning it is already sorted. Lastly in the main function, it declares an array of size 10 and a user input for the user to enter 10 integers. Calls the mergeSort to sort the array and prints the sorted array.

Output:

```

Enter 10 integers:
15 13 16 20 14 51 31 61 2 41

Sorted array:
2 13 14 15 16 20 31 41 51 61

```